MIDTERM PROJECT ASSIGNMENT

Fault-Tolerant University Information System

Course: Fault Tolerance and Dependable Computing

1. Project Overview

The purpose of this project is to design, implement, and evaluate a dependable university information system that can continue operating when hardware, software, or service failures occur. Students will apply the concepts of fault tolerance, reliability engineering, hardware redundancy, and software fault tolerance in one integrated project.

The project should model a realistic university platform with several critical services, such as student registration, tuition/payment processing, transcript generation, timetable generation, and academic record management.

2. Learning Objectives

- Explain fundamental concepts of fault tolerance, dependability, reliability, and availability.
- Identify and analyze possible system failures and their effects.
- Calculate and interpret reliability-related metrics such as MTTF, MTBF, MTTR, and availability.
- Apply hardware fault-tolerance mechanisms such as redundancy, replication, ECC, and RAID.
- Implement software fault-tolerance techniques including retries, timeouts, checkpointing, rollback, and graceful degradation.
- Design experiments that demonstrate system behavior before and after fault-tolerance mechanisms are introduced.
- Evaluate whether the resulting system meets defined reliability and availability requirements.

3. Project Scope

The project must cover the following four course topics:

Topic	Required Focus	Expected Output
1. Introduction to Fault Tolerance and Dependable Computing	Dependability concepts, faults, errors, failures, fault models, availability, and system requirements.	System dependability model and failure scenarios.
2. Reliability Engineering and Failure Analysis	MTTF, MTBF, MTTR, availability, failure rates, fault-tree or failure analysis.	Reliability calculations and failure-analysis report.
3. Hardware Fault Tolerance	Redundancy, replication, RAID/ECC concepts, backup infrastructure, and single points of failure.	Hardware/infrastructure fault-tolerance design.
4. Software Fault Tolerance	Retries, timeouts, circuit breakers, checkpointing, rollback/recovery, idempotency, replication, and graceful degradation.	Fault-tolerant application implementation and experiments.

4. System Requirements

1. Develop a baseline version of the system without advanced fault-tolerance mechanisms.
2. Identify at least five realistic failure scenarios.
3. Include at least three independent services or modules.
4. Demonstrate at least two hardware/infrastructure-level fault-tolerance mechanisms.
5. Demonstrate at least four software-level fault-tolerance mechanisms.
6. Collect failure and recovery data during experiments.
7. Compare baseline and fault-tolerant versions using measurable reliability and availability indicators.
8. Document all assumptions, architecture decisions, experiments, and results.

5. Suggested System Architecture

A recommended architecture is a small distributed university platform consisting of:

- API Gateway / Load Balancer
- Student Service
- Payment Service
- Academic Records / Transcript Service
- Timetable Service
- Database
- Monitoring and Logging Service

Students may use Python, Java, C#, Go, or another suitable language. Containerization with Docker is recommended, and Kubernetes may be used for replication, health checks, self-healing, and rolling updates.

6. Required Failure Scenarios

Failure	Simulation	Expected Observation
Application crash	Terminate one application instance and observe recovery.	Record failure behavior, detection time, recovery time, and data consistency.
Database failure	Make the database temporarily unavailable and evaluate recovery.	Record failure behavior, detection time, recovery time, and data consistency.
Network/service timeout	Introduce delayed or failed service-to-service communication.	Record failure behavior, detection time, recovery time, and data consistency.
Hardware/node failure	Simulate failure of one server or Kubernetes node.	Record failure behavior, detection time, recovery time, and data consistency.
Corrupted or lost transaction	Simulate an interrupted payment or academic-record operation.	Record failure behavior, detection time, recovery time, and data consistency.
High load	Generate a high number of concurrent requests and evaluate service degradation.	Record failure behavior, detection time, recovery time, and data consistency.

7. Reliability Engineering Analysis

Students must calculate and discuss:

- MTTF (Mean Time To Failure)
- MTBF (Mean Time Between Failures)
- MTTR (Mean Time To Repair/Recover)
- System availability
- Observed failure rate
- Recovery time
- Number of failed requests and successfully recovered requests

Where appropriate, students should compare theoretical calculations with experimental measurements and explain discrepancies.

8. Hardware Fault-Tolerance Component

The project must explain and demonstrate how infrastructure redundancy reduces the impact of component failures. Depending on the selected architecture, students may use:

- Multiple application instances / server replicas
- Database replication
- RAID concepts or simulated storage redundancy
- ECC as a documented hardware mechanism
- Backup infrastructure
- Load balancing
- Failure of one node without total service interruption

9. Software Fault-Tolerance Component

Implement at least four of the following:

- Retry with exponential backoff
- Timeouts
- Circuit breaker
- Idempotent transaction processing
- Checkpointing
- Rollback and recovery
- Health checks
- Graceful degradation
- Duplicate-request detection
- Service replication

10. Experimental Evaluation

Run controlled experiments on the baseline and fault-tolerant systems. For each experiment, record the failure injected, system state, detection time, recovery time, successful requests, failed requests, and data-consistency result.

Experiment	Failure	Detection Time	Recovery Time	Failed Requests	Result

11. Report Structure

1. Introduction and project motivation
2. System requirements and assumptions
3. Dependability and fault model
4. Reliability and failure analysis
5. System architecture
6. Hardware fault-tolerance design
7. Software fault-tolerance design
8. Implementation
9. Experimental methodology
10. Results and comparison
11. Discussion and limitations
12. Conclusion
13. References
14. Appendix: source code, configuration, logs, and additional results

12. Deliverables

- Source code repository
- System architecture diagram
- Baseline implementation
- Fault-tolerant implementation
- Failure-injection / testing scripts
- Experimental dataset or logs
- Final technical report
- Short demonstration of at least three failure-and-recovery scenarios

13. Assessment Rubric

Criterion	Weight	Description
Dependability and failure analysis	15%	Correct identification and analysis of faults, errors, and failures.
Reliability calculations	15%	Correct MTTF, MTBF, MTTR, availability, and failure analysis.
Hardware fault tolerance	15%	Appropriate redundancy and infrastructure design.
Software fault tolerance	20%	Correct implementation of recovery and resilience mechanisms.
Experiments and evaluation	20%	Controlled failure injection, measurements, and comparison.
Report and documentation	10%	Clear technical explanation, architecture, results, and references.
Demonstration	5%	Successful demonstration of failure and recovery.

14. Recommended Technology Stack

- Python + FastAPI for services
- Docker for containerization
- Kubernetes for service replication and self-healing
- PostgreSQL or SQLite for data storage
- Prometheus and Grafana for monitoring
- Python scripts for workload generation and failure injection
- Git/GitHub or another version-control platform

15. Project Success Criteria

The project is considered successful when the team can demonstrate that selected failures are detected, their impact is limited, and affected services recover without unnecessary loss or duplication of data. The final evaluation should be supported by measured experimental evidence rather than only architectural claims.